

CRUD de Games em Flutter

Guia detalhado de arquitetura, configuração, SQLite, UI/UX e fluxo
CRUD

Projeto: CRUD-de-Games---Educat

Stack principal: Flutter + Dart + Material 3 + SQLite (sqflite)

Objetivo: Explicar os principais elementos da aplicação, as configurações necessárias para executar o projeto e a função de cada camada do código.

Sumário

1. Visão geral da aplicação
2. Requisitos e configuração do ambiente
3. Dependências do projeto
4. Estrutura de pastas e arquitetura
5. Ponto de entrada e tema visual
6. Modelo de dados Game
7. Persistência local com SQLite
8. Tela principal e listagem
9. Formulário de cadastro e edição
10. Fluxos Create, Read, Update e Delete
11. Técnicas de UI/UX aplicadas
12. Como executar o projeto
13. Como testar o CRUD
14. Erros comuns e solução de problemas
15. Melhorias recomendadas

1. Visão geral da aplicação

O GameShelf é um mini projeto educacional em Flutter para controlar uma coleção de jogos. A aplicação funciona de forma offline e armazena os registros em um banco SQLite local no próprio dispositivo.

- Cadastrar jogos com nome, plataforma, status, nota e observações.
- Listar todos os jogos salvos no dispositivo.
- Editar registros já existentes.
- Excluir registros mediante confirmação.
- Exibir feedback visual após cadastrar, alterar ou excluir.
- Organizar visualmente os jogos por status, usando cor e ícone como apoio.

Campos armazenados

Campo	Tipo Dart	Tipo SQLite	Finalidade
id	int?	INTEGER	Identificador único e chave primária.
name	String	TEXT	Nome do jogo.
platform	String	TEXT	Plataforma do jogo.
status	String	TEXT	Quero jogar, Jogando ou Finalizado.
rating	double?	REAL	Nota opcional entre 0 e 10.
notes	String	TEXT	Observações livres.

2. Requisitos e configuração do ambiente

Antes de executar o projeto, é necessário ter um ambiente Flutter funcional.

- Flutter SDK instalado e disponível no PATH.
- Dart fornecido pelo próprio Flutter SDK.
- Android Studio ou VS Code com extensões Flutter/Dart.
- Para Android: Android SDK, emulador ou dispositivo físico com depuração USB.
- Para iOS: macOS com Xcode e CocoaPods.
- Git para clonar o repositório.

Verificação do ambiente

```
flutter --version  
flutter doctor
```

O comando flutter doctor identifica problemas como Android SDK ausente, licenças não aceitas, Xcode incompleto ou falta de dispositivos disponíveis.

Clone do repositório

```
git clone https://github.com/guerradiegop/CRUD-de-Games---Educacional.git
cd CRUD-de-Games---Educacional
```

Importante: Como o repositório foi criado com foco no código educacional, caso as pastas nativas do Flutter não existam, execute ``flutter create .`` na raiz do projeto.

3. Dependências do projeto

O arquivo pubspec.yaml declara o SDK compatível e os pacotes usados pela aplicação.

```
environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  sqflite: ^2.4.2
  path: ^1.9.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
```

Função de cada dependência

- flutter: framework principal responsável pelos widgets, navegação, tema e renderização da interface.
- cupertino_icons: conjunto de ícones do estilo Cupertino; é dependência padrão em muitos projetos Flutter.
- sqflite: oferece acesso ao SQLite em aplicativos Flutter, incluindo criação de banco, consultas, inserções, atualizações e exclusões.
- path: facilita a composição segura do caminho do arquivo do banco de dados em diferentes sistemas.
- flutter_test: infraestrutura oficial para testes automatizados em Flutter.
- flutter_lints: conjunto de regras estáticas para manter o código mais consistente.

Instalação dos pacotes

```
flutter pub get
```

4. Estrutura de pastas e arquitetura

```
lib/  
├── database/  
│   └── database_helper.dart  
├── models/  
│   └── game.dart  
├── pages/  
│   ├── home_page.dart  
│   └── game_form_page.dart  
└── main.dart
```

A arquitetura é deliberadamente simples. Ela separa modelo, persistência e interface sem introduzir camadas complexas de estado, repositórios ou injeção de dependência. Isso facilita o estudo de CRUD e SQLite.

- **models:** representa os dados do domínio da aplicação.
- **database:** concentra o acesso ao SQLite.
- **pages:** contém as telas e a lógica de interação com o usuário.
- **main.dart:** inicializa a aplicação e define o tema global.

5. Ponto de entrada e tema visual

O arquivo `lib/main.dart` inicializa o Flutter e cria o widget raiz da aplicação.

```
void main() {  
  WidgetsFlutterBinding.ensureInitialized();  
  runApp(const GamesApp());  
}
```

`WidgetsFlutterBinding.ensureInitialized()` garante que os serviços internos do Flutter estejam inicializados antes de qualquer operação que dependa da engine.

MaterialApp

- `debugShowCheckedModeBanner: false` remove a faixa DEBUG.
- `title` define o nome lógico da aplicação.
- `useMaterial3: true` habilita Material Design 3.
- `ColorScheme.fromSeed` gera uma paleta coerente a partir de uma cor-base.
- `scaffoldBackgroundColor` estabelece um fundo suave e neutro.
- `cardTheme` padroniza os cards com cantos arredondados e sem elevação excessiva.

- `inputDecorationTheme` padroniza os campos do formulário.

```
final colorScheme = ColorScheme.fromSeed(  
  seedColor: const Color(0xFF6650A4),  
  brightness: Brightness.light,  
);
```

UI/UX: O uso de um tema global reduz inconsistências visuais e evita repetir configurações de estilo em cada widget.

6. Modelo de dados Game

O arquivo `lib/models/game.dart` representa uma linha da tabela `games`.

```
class Game {  
  final int? id;  
  final String name;  
  final String platform;  
  final String status;  
  final double? rating;  
  final String notes;  
}
```

Por que `id` e `rating` são opcionais?

O `id` é nulo antes de um registro ser inserido, porque o SQLite gera o valor automaticamente com `AUTOINCREMENT`. A nota também é opcional para permitir cadastrar um jogo ainda não avaliado.

Conversão objeto ↔ banco

```
Map<String, dynamic> toMap() {  
  return {  
    'id': id,  
    'name': name,  
    'platform': platform,  
    'status': status,  
    'rating': rating,  
    'notes': notes,  
  };  
}
```

`toMap()` transforma o objeto `Game` em um mapa compatível com `sqlite`. O construtor `factory Game.fromMap` faz o processo inverso.

```
factory Game.fromMap(Map<String, dynamic> map) {  
  return Game(  
    id: map['id'] as int?,
```

```

name: map['name'] as String,
platform: map['platform'] as String,
status: map['status'] as String,
rating: map['rating'] != null
    ? (map['rating'] as num).toDouble()
    : null,
notes: map['notes'] as String? ?? "",
);
}

```

7. Persistência local com SQLite

A classe DatabaseHelper implementa o acesso ao banco. Ela usa uma instância única (singleton) e mantém uma referência ao objeto Database para evitar abrir o mesmo banco repetidamente.

```

class DatabaseHelper {
    DatabaseHelper._();

    static final DatabaseHelper instance = DatabaseHelper._();
    static Database? _database;
}

```

Inicialização do banco

```

final databasePath = await getDatabasesPath();
final path = join(databasePath, 'games.db');

return openDatabase(
    path,
    version: 1,
    onCreate: (db, version) async {
        // criação das tabelas
    },
);

```

getDatabasesPath() retorna o diretório apropriado para bancos no sistema operacional. O pacote path monta o caminho de forma portátil, evitando concatenar barras manualmente.

Tabela games

```

CREATE TABLE games (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    platform TEXT NOT NULL,
    status TEXT NOT NULL,
    rating REAL,
    notes TEXT NOT NULL DEFAULT ""
)

```

Operações CRUD no banco

Operação	Método	sqlite
Create	insertGame(Game)	db.insert
Read	getGames()	db.query
Update	updateGame(Game)	db.update
Delete	deleteGame(int)	db.delete

Uso de whereArgs

```
return db.update(  
    'games',  
    data,  
    where: 'id = ?',  
    whereArgs: [game.id],  
);
```

Os placeholders '?' combinados com whereArgs evitam construir cláusulas SQL por concatenação direta, além de deixar a consulta mais clara.

8. Tela principal e listagem

A HomePage é um StatefulWidget porque a lista de jogos e o estado de carregamento mudam ao longo do uso.

```
List<Game> _games = [];  
bool _loading = true;
```

Carregamento inicial

```
@override  
void initState() {  
    super.initState();  
    _loadGames();  
}
```

_loadGames() consulta o banco e chama setState para redesenhar a interface com os dados carregados. A verificação mounted evita atualizar um widget que já tenha saído da árvore.

Estados da interface

- Carregando: CircularProgressIndicator.
- Sem registros: empty state com ícone, mensagem explicativa e botão de ação.
- Com registros: ListView.separated com cards.

- Atualização manual: RefreshIndicator permite gesto de pull-to-refresh.

Card de jogo

Cada card mostra nome, plataforma, status, nota e observações. O card inteiro é clicável para edição e também há um PopupMenu com ações explícitas de editar e excluir.

Hierarquia visual: Nome do jogo recebe maior peso tipográfico. Plataforma e observações usam cores de menor contraste, enquanto status e nota funcionam como metadados de leitura rápida.

9. Formulário de cadastro e edição

A mesma tela GameFormPage serve tanto para inclusão quanto para alteração. A presença de widget.game determina o modo.

```
bool get isEditing => widget.game != null;
```

Controllers

TextEditingController mantém e recupera os valores digitados nos campos. Eles são inicializados no initState e liberados no dispose.

- _nameController: nome do jogo.
- _platformController: plataforma.
- _ratingController: nota.
- _notesController: observações.

Validação

- Nome é obrigatório.
- Plataforma é obrigatória.
- Nota pode ficar vazia.
- Quando informada, a nota deve ser numérica e permanecer entre 0 e 10.
- Vírgula é convertida em ponto para aceitar entrada decimal comum em português.

```
final rating = double.tryParse(
  value.replaceAll(',', '.'),
);

if (rating == null) return 'Informe uma nota válida';
if (rating < 0 || rating > 10) {
  return 'A nota deve estar entre 0 e 10';
}
```

Botão de salvar

Durante a gravação, `_saving` desabilita o botão e substitui o ícone por um indicador de progresso. Isso reduz o risco de toques repetidos e comunica que a ação está sendo processada.

10. Fluxos Create, Read, Update e Delete

Create - cadastrar

- Usuário toca em Novo jogo.
- `HomePage` abre `GameFormPage` sem um objeto `Game`.
- O formulário é validado.
- Um novo `Game` é criado com `id` nulo.
- `insertGame` grava os campos no `SQLite`.
- A tela fecha retornando `true`.
- `HomePage` recarrega a lista e mostra `SnackBar` de sucesso.

Read - listar

- `HomePage` executa `_loadGames` no `initState`.
- `DatabaseHelper.getGames` consulta a tabela `games`.
- Cada mapa retornado é convertido por `Game.fromMap`.
- Os objetos são armazenados em `_games` e renderizados em `cards`.

Update - editar

- Usuário toca no card ou escolhe Editar.
- `GameFormPage` recebe o objeto existente.
- `Controllers` são preenchidos com os valores atuais.
- Ao salvar, `updateGame` altera a linha cujo `id` corresponde ao jogo.
- A `HomePage` recarrega os dados.

Delete - excluir

- Usuário escolhe Excluir.
- Um `AlertDialog` pede confirmação.
- Somente após confirmação, `deleteGame` executa a exclusão usando o `id`.
- A lista é recarregada e um `SnackBar` confirma a remoção.

11. Técnicas de UI/UX aplicadas

Técnica	Benefício
Material 3	Componentes modernos e consistência visual.
Tema global	Reduz variações de cor, borda e estilo entre

FAB estendido	telas.
Empty state	Torna a ação principal 'Novo jogo' altamente visível.
Feedback de carregamento	Evita tela vazia e explica o próximo passo.
SnackBar	Comunica que o sistema está trabalhando.
Confirmação de exclusão	Confirma ações concluídas sem interromper o fluxo.
Validação inline	Previne perda acidental de dados.
Cores de status	Mostra o erro junto ao campo que precisa ser corrigido.
Ícones + texto	Acelera o reconhecimento visual.
Card clicável	Evita depender exclusivamente de cor.
Menu contextual	Amplia a área de toque para editar.
	Mantém ações secundárias disponíveis sem poluir o layout.

12. Como executar o projeto

Passo 1 - preparar estrutura Flutter

```
flutter create .
flutter pub get
```

Passo 2 - verificar dispositivos

```
flutter devices
```

Passo 3 - executar

```
flutter run
```

Android

- Abra um emulador no Android Studio ou conecte um telefone com depuração USB.
- Aceite licenças Android, se necessário, com flutter doctor --android-licenses.
- Execute flutter run.

iOS

- Disponível somente em macOS.
- Instale Xcode e CocoaPods.
- Abra simulador ou conecte um dispositivo.
- Execute flutter run.

Build Android

```
flutter build apk  
# ou  
flutter build appbundle
```

Análise estática

```
flutter analyze
```

13. Como testar o CRUD

Um roteiro simples de validação manual pode ser usado após iniciar a aplicação.

Caso	Ação	Resultado esperado
Cadastro válido	Criar um jogo com nome e plataforma.	Registro aparece na lista.
Validação de nome	Salvar com nome vazio.	Mensagem de campo obrigatório.
Nota inválida	Informar 12.	Validação impede salvamento.
Edição	Alterar status e nota.	Card reflete os novos valores.
Exclusão cancelada	Abrir excluir e cancelar.	Registro permanece.
Exclusão confirmada	Confirmar exclusão.	Registro desaparece e Snackbar é exibido.
Persistência	Fechar e abrir o app.	Dados continuam disponíveis.

14. Erros comuns e solução de problemas

flutter: command not found

O Flutter SDK não está corretamente incluído no PATH. Revise a instalação e reinicie o terminal.

No devices found

Inicie um emulador/simulator ou conecte um dispositivo físico. Confirme com flutter devices.

Pastas android/ios ausentes

Execute flutter create . na raiz do repositório para gerar a estrutura de plataforma sem substituir a pasta lib.

Dependências não resolvidas

```
flutter clean
flutter pub get
```

Banco com estrutura antiga durante desenvolvimento

Como o banco usa version: 1 e somente onCreate, alterar o CREATE TABLE depois que o banco já existe não atualiza automaticamente o arquivo existente. Em desenvolvimento, você pode desinstalar o app/limpar dados. Em produção, crie migrações com onUpgrade e aumente a versão do banco.

Erro ao salvar depois de fechar a tela

O projeto já usa verificações mounted em pontos importantes. Essa prática evita chamar setState, Navigator ou ScaffoldMessenger quando o State não está mais ativo.

15. Melhorias recomendadas

- Adicionar busca por nome e filtro por status/plataforma.
- Criar tela de detalhes separada da edição.
- Usar enum para status em vez de strings livres.
- Adicionar onUpgrade para migrações de banco.
- Criar camada Repository para desacoplar UI de SQLite.
- Adotar gerenciador de estado como Provider, Riverpod ou Bloc em uma versão maior.
- Adicionar testes unitários para Game e DatabaseHelper.
- Adicionar testes de widgets para formulário e lista.
- Implementar modo escuro com ThemeMode.
- Adicionar ordenação por nota, nome ou status.
- Permitir capa/imagem do jogo armazenando caminho local da imagem.
- Criar exportação/importação dos dados.

Sugestão de evolução arquitetural

```
UI (Pages)
↓
State / Controller
↓
Repository
↓
```

Data Source (SQLite)



Database

A arquitetura atual é adequada para um mini projeto educacional. A estrutura acima passa a ser interessante quando o projeto cresce, há múltiplas fontes de dados ou a lógica de negócio começa a ficar maior.

Conclusão

O projeto demonstra de forma objetiva um CRUD completo em Flutter com SQLite local. Ele cobre modelagem de dados, persistência, navegação, formulários, validação, feedback ao usuário, estados de interface e princípios importantes de UI/UX. Por ser enxuto, também funciona bem como base para aulas, exercícios de laboratório e projetos de extensão.

Repositório: <https://github.com/guerradiegop/CRUD-de-Games---Educacional>